Faculty of Engineering Sciences

Heidelberg University

Master's Thesis

in Computer Engineering

submitted by

Bc. VLADISLAV VÁLEK

born in Šumperk, Czech Republic

2026

# SHARED CPU FPGA FILESYSTEM ACCESS ON FAST NVME DEVICES

This Master's Thesis has been carried out by

Bc. Vladislav Válek

at the

Institute of Computer Engineering (ZITI)

under the supervision of

Prof. Dr. Dirk Koch

## ABSTRACT

Abstract in English.



## KEYWORDS

PCIe, PCI Express, DMA, Direct Memory Access, host-bypassing, peer-to-peer, SPDK, NVMe, FPGA



Typeset by the `thesis` package, version 4.13; `https://latex.fekt.vut.cz/`

# Erklärung

Ich versichere, dass ich diese Arbeit selbstständig verfasst habe und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Heidelberg ..... .....

Unterschrift

# ACKNOWLEDGEMENT

I would like to thank my supervisor, Prof. Dr. Dirk Koch for his valuable feedback when writing this thesis as well as in my research work at the Heidelberg University. I would also like to thank my colleague, Dr. Riadh Ben Abdelhamid, for providing necessary user feedback for the current course of research. Further, I give many thanks to my colleagues at CESNET, a.l.e., Ing. Jakub Cabal and Ing. Martin Špinler, for mentoring me in the development of PCIe-compliant IPs and software. Finally, I thank my mom, Ing. Marta Válková, for supporting me throughout my studies.

# Contents

**Introduction** **19**

**1 PCI Express** **21**
1.1 Communication model . . . . . 21
1.2 Link configuration . . . . . 23
1.3 Device configuration . . . . . 25
1.4 Peer-to-peer transfer . . . . . 27

**2 Integrated core for PCIe on FPGAs** **29**
2.1 Interfaces to user logic . . . . . 29

**3 Non-volatile Memory Express** **31**
3.1 Principles of operation . . . . . 33
3.1.1 Command submission . . . . . 34
3.1.2 Command completion . . . . . 35
3.1.3 Ordering . . . . . 37
3.2 Multi-pathing . . . . . 37

**4 DMA Iuventus** **39**
4.1 TLP Headers . . . . . 39
4.2 Software configuration . . . . . 41
4.2.1 Memory partitioning . . . . . 42
4.2.2 Queue initialization . . . . . 43
4.3 Communication model . . . . . 44
4.3.1 Read command . . . . . 44
4.3.2 Write command . . . . . 45
4.4 Hardware implementation . . . . . 46
4.5 Evaluation . . . . . 46

**Conclusion** **47**

**Symbols and abbreviations** **51**

**List of appendices** **55**

**A SPDK Framework configuration** **57**

# List of Figures


1.1 Example of PCIe topology . . . . . 22
1.2 Examples of PCIe transports . . . . . 23
1.3 Various types of PCIe CEM connectors . . . . . 24
1.4 PCIe topology with different link widths . . . . . 25
1.5 Influence of the ACS mechanism . . . . . 27
3.1 Elementary block scheme of the NVMe device . . . . . 31
3.2 Initialized system with one NVMe device and a CPU . . . . . 32
3.3 Two examples of an empty queue . . . . . 33
3.4 Two examples of a full queue . . . . . 33
3.5 Format of Submission Queue Entry . . . . . 34
3.6 Format of Completion Queue Entry . . . . . 36
3.7 Example topology with M:N mesh network . . . . . 38
4.1 Format of the header used on the Requester Request interface . . . . 39
4.2 Format of the header used on the Completer Request interface . . . . 40
4.3 Format of the header used on the Completer Completion interface . . 41

# List of Tables

1.1 PCIe throughput based on its versions . . . . . 24

# List of Algorithms

# List of Listings

# Introduction

Here comes the introduction of the thesis, for example...

This thesis is devoted to **DSP!** (**DSP!**), especially it analyses the effect happening when the Nyquist condition for ***symfs!*** (**symfs!**) is not satisfied.[1]

The template is set to twoside printing by default. Do not be surprised that you find empty pages in your PDF. They are there to make the chapters and other important stuff begin on the right side when the document is printed. Having a serious reason to print one-sided, please switch the option `twoside` to `oneside`!

[1] This sentence is only to demonstrate how abbreviations can be used and typeset.

# 1 PCI Express

The *Peripheral Component Interconnect Express* (PCIe) is the interconnect standard that emerged in 2003 from the *Peripheral Component Interconnect* (PCI) standard. The PCIe addressed the drawbacks of the PCI that became increasingly challenging when more devices and more throughput were required. These include:

- Shared-bus topology that, in spite of its simplicity, was rendered unscalable since with increasing amount of devices, the bus gets longer which weakens the signal for the farthest device.
- Common clock which does not fit well with parallel data transmission model especially when lane skew takes place, meaning the bits of data travel unevenly fast over the wires.
- Half-duplex communication allows only one device to communicate at one point at a time.
- Reflected-wave signaling which was used to reduce the power consumption on the bus. This uses an interference of the sent wave from the master (which had driven the signal to the lower voltage than needed for the digital circuits on the bus) and its reflection that originated on the end of a bus (there is no termination on the bus' end). By the addition of these two waves, the signal reaches the required voltage level for the digital circuits[1].

The PCI protocol did not keep pace with increasing demand of computer peripheral communication structures and needed to be changed. The PCI Express protocol significantly changed the strategy for both the communication topology and the physical properties of the interconnect[1].

## 1.1 Communication model

The basic architecture is shown in fig. 1.1. Connections are implemented as point-to-point links between ports on different device types organized in a tree. The center of the topology is the *Root Complex* (RC), which provides *Root Ports*. Each device communicating over the infrastructure (such as today's GPU/FPGA accelerators) is called *Function* and is the major source of traffic within the system[1]. Each Function contains an *Upstream port* for exchanging data. Since the number of Root Ports is limited by the processor's interconnect, fan-out is provided by *PCIe Switches*, which supply multiple *Downstream ports* and a single Upstream port. Each port supports full-duplex communication and introduces asynchronous clocking model. Using this model significantly improves the possible throughput of the interconnect where clock gets reconstructed from the patterns in the incoming data on the receiving port. Using Switches, the number of devices can be increased arbitrarily, making PCIe

a switched network[2]. Although the PCIe specification introduces many advanced features, this thesis presents mostly the necessary ones for its purposes.

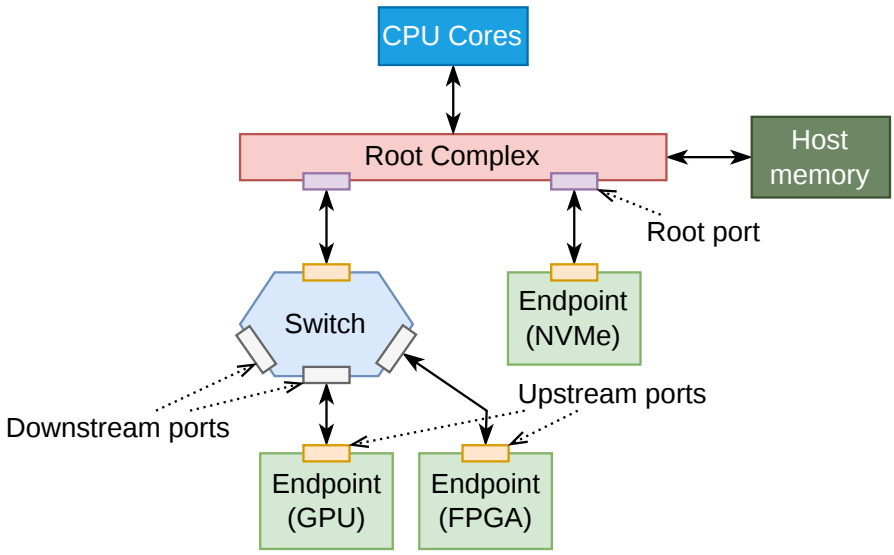


Fig. 1.1: Example of a topology using PCIe interconnect with its designated elements

Devices communicate using units called *Transaction Layer Packets* (TLPs), forming a packetized transfer. Each TLP contains a PCIe header used for identification followed by an optional payload. There are many TLP types, but the ones mentioned in this work are *Memory Read Request* (MRd) and *Memory Write Request* (MWr). Requests may require a response (called *non-posted*, as with MRds) in the form of *Completion* (Cpl) presenting a third TLP type. Such requests are called *non-posted* (like MRds) while requests that do not require a response are called *posted* (like MWrs). The device creating a request is called *Requester*, while the device 'completing' the request (sending a response or just accepting the payload) is called *Completer*.

The RC is located on the CPU die and tightly integrated with the processing cores to provide fast access to the system's I/O devices. The RC also provides the connection to host memory, which is the primary communication path used by DMA engines for *Host-to-Device* (H2D) and *Device-to-Host* (D2H) transfers.

[1]The PCIe specification (with concordance to the PCI) defines three *System elements* to identify its devices, namely *Bus*, *Device* and *Function*. Every point-to-point connection in the topology makes a separate Bus together with internal (virtual) buses of the Switches and the Root Complex. The Device identifier stays as a remnant of the PCI standard and every Bus within the PCIe topology contains only one. Within each Device, one or multiple Functions can be present. The Function is basically and addressable entity in the configuration space where *Endpoint* is the most important type of a Function. Using the term 'device' in this thesis will refer to a general device in the PCIe topology that can contain one or multiple Functions (or Endpoints).

Multiple Root Ports or Downstream ports enable direct *Device-to-Device* (D2D) communication, as shown in fig. 1.2.

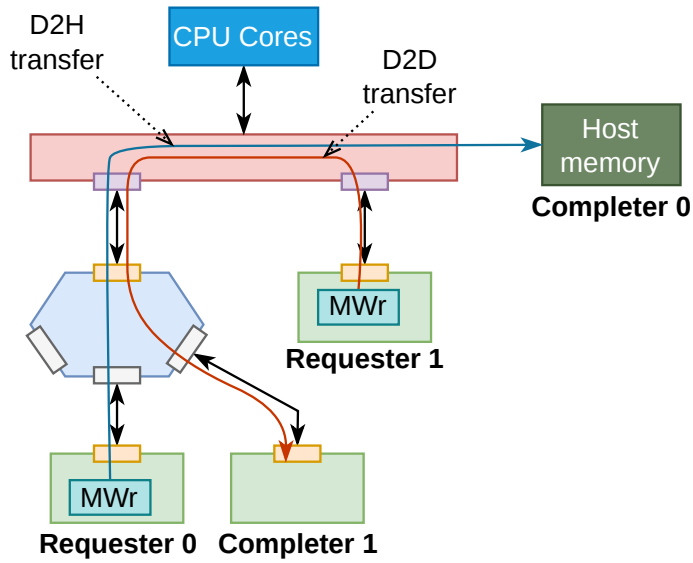


Fig. 1.2: Examples of the transports occurring in the PCIe domain. Each device can be both Requester (i.e. dispatching a request) and a Completer (i.e. receiving a request) at the same time.

## 1.2 Link configuration

The PCIe specification gets published in versions called *generations*. With every new specification, the throughput of the bus gets doubled while keeping backward and forward compatibility between its standards (this is an important feature of PCIe). This is the reason, why the physical implementation of the protocol stack remains so complicated, especially on the physical layer. The physical connection between two ports consists of multiple differential pairs called *lanes* that are bundled together into a *link*. Each lane allows for full-duplex communication, where each direction is communicated using a separate differential pair. The link can either contain 1, 2, 4, 8 or 16 lanes. The communication speeds are shown in table 1.1.

As for the physical attachment interface, this thesis works with two of them, namely *Card Electromechanical* (CEM) and *M.2*. The CEM standard is the oldest one used for *Add-in Cards* with GPU chips, FPGA accelerators or *Network Interface Cards* (NICs). It supports from 1 to 16 lanes with with wider link widths supporting lower ones and vice versa (these however with specially carved connectors that fit longer connector of the attached card). This form-factor allows for connection of the most power consuming peripherals where the connector can supply up to 75 W.

Table 1.1: PCIe throughput based on its versions. The throughput is calculated without the overhead of the Physical layer

| Version | Line code | | Link throughput [GBps] | | | | |
|---|---|---|---|---|---|---|---|
| | Electrical | Logical | x1 | x2 | x4 | x8 | x16 |
| Gen3 | NRZ | 128b/130b | 0.985 | 1.969 | 3.938 | 7.877 | 15.754 |
| Gen4 | NRZ | 128b/130b | 1.969 | 3.938 | 7.877 | 15.754 | 31.508 |
| Gen5 | NRZ | 128b/130b | 3.938 | 7.877 | 15.754 | 31.508 | 63.015 |
| Gen6 | PAM-4 | 242B/256B | 7.563 | 15.125 | 30.25 | 60.5 | 121 |
| Gen7 | PAM-4 | 242B/256B | 15.125 | 30.25 | 60.5 | 121 | 242 |

The accelerators can require additional power by introducing external power supply attachments. The most common ATX connector adds another 75 W for its 6-pin configuration or 150 W for the 8-pin configuration[3].

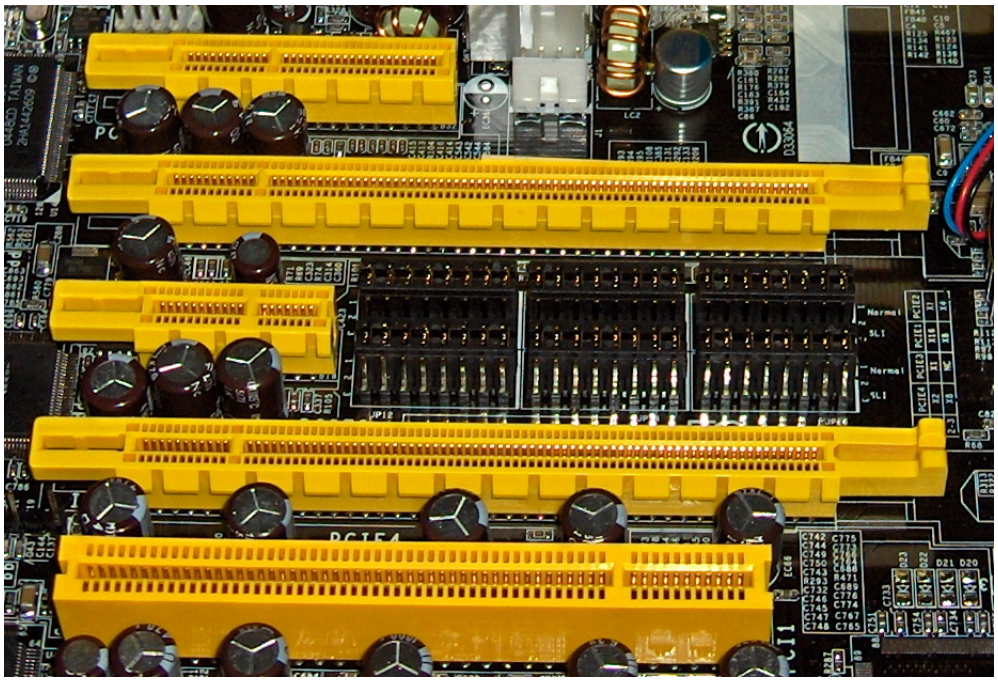

Fig. 1.3: Different types of CEM PCIe connectors supporting 4, 16, 1 or 16 lanes (looking from the top). The bottom connector belongs to the old PCI bus[4]

The second type of physical attachment, *M.2*, is designed for ultra-light, power efficient platforms such as wireless modules or (as in the case of this thesis) *Non-volatile Memory Express* (NVMe) drives. This connector has a fixed set of 4 lanes and a variable module length ranging from 30 mm to 110 mm. These connectors present a detachable form factor for PCIe links but the standard allows for permanently attached devices to be connected as well.

As it is obvious from the variety of link configurations in terms of speeds and the amount of lanes, the multitude of PCIe devices inside a compute node needs to agree on common parameters in order to facilitate data transfer. This process of negotiation is called *link training* and it takes place after the network get powered up. The training is done on per-link-basis so there are multiple differently configured links in the system at a given point in time. This is presented on fig. 1.4 where the M.2 SSD needs to communicate with the processor (where Root Ports can usually utilize up to 16 lanes) or with a NIC which supports up to 16 lanes. Other than link's lane count and transfer rate, other configured parameters include lane polarity and lane reversal. Since the training and the negotiated link parameters are tha matter of the physical layer, the link properties remain transparent for the user.

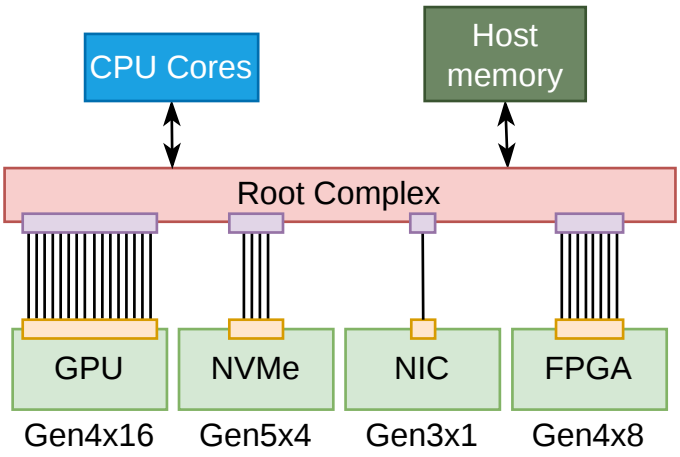


Fig. 1.4: Topology with multiple interconnected devices where each supports different link width.

## 1.3 Device configuration

After the initial link training is done and every link is active, the host system does device discovery and configuration upon system boot in order to map the network. The process of discovery is called *enumeration* and involves a CPU communicating through the Root Complex in order to find every connected device in the topology and assign its BDF identifier (this includes assigning numbers also to buses between ports that do not necessarily connect to an Endpoint, like two Switches or a Switch with the Root Complex). The search is done in a depth-first way where each device is registered if it responds with a valid *Vendor ID* that is located within device's configuration registers. After assigning all BDFs in the topology, the configuration software further configures registers on the discovered devices. The configuration is

done solely by the Root Complex in order to avoid the complexity of management when multiple devices (e.g. Endpoints) would try to configure each other. The initial assigning of configuration attributes is done using an IO-based transfer but further setting is done using *Memory-mapped I/O* (MMIO).

Each Endpoint contains configuration registers in a 64-byte *PCI Configuration Header*, which holds the most important attributes controlling the Function's ability to initiate transactions over PCIe and to be addressable within a host system. Firstly, since devices cannot generate Requests on their own by default, the *Bus Master* attribute enables this feature. Secondly, in order for adjacent devices to locate the current Function and to access its internal, user-defined registers/memory space, at least one of the *Base Address Registers* (BARs) must be initialized. By using BARs, each Endpoint requests a memory space within a host system for MMIO access. This is needed to access the application logic from the outside since each Endpoint accepts only those transactions whose addresses are in the ranges specified by its BARs. Each Function can register up to six 32-bit BARs which provides memory segmentation needed by some host applications. When 64-bit addressing is requested, two consecutive BARs are used. Throughout this work, the observation has been made that most devices allocate one to two BARs and, when not exceeding 4 GiB of space, an operating system maps them to 32-bit address ranges even when 64-bit addressing has been requested. First of all, this addressing has to be enabled in the UEFI settings of the host system.

Besides than the sheer capability for devices to be localizable in the system, other parameters are configured and need to be abided by by the user applications (e.g. the user-defined FPGA logic). A function defines its supported *Maximum Payload Size* (MPS) in the *Device Capabilities register*. Seeing this value, the configuration software sets its preferred value in a field in the *Device Control register*. The value of this field prohibits the Transmitter to dispatch TLP with greater payload than MPS specified and the Receiver to process them. The configuration allows further adjustment using the *Maximum Read Request Size* (MRRS) in the *Device Control register* which prohibits the Transmitter to dispatch a Read Request greater than the specified size. Available values for these two parameters range from 128 to 4096 bytes (in powers of two steps). The configuration software balances these parameters based on latency/bandwidth requirements in the network. The configuration has to take account of the pairs of adjacent devices communicating with each other since receiving a TLP on a function's port whose size is larger than its set MPS results in the function rejecting this packet and reporting a fatal error[2].

[2]The PCIe standard does not require System Elements to perform segmentation of large TLPs.

## 1.4 Peer-to-peer transfer

An important aspect of PCIe (with its predecessor as well) and the structure of its network allowed to introduce *Direct Memory Access* (DMA) that allows to copy large amounts of data without the host CPU facilitating such copying. The usual approach it the exchange of data between PCIe devices and the host CPU through buffers in the host memory. However, the network infrastructure to address other devices in the topology as well. This type of D2D transport is called *Peer-to-peer* (P2P) or *Host Bypassing* and makes a fundamental part of this thesis. There are several implications of running such transport in the host system. The specification makes the support for each Switch mandatory (for transport between its Downstream ports) but optional for the Root Complex (for transport between its Root Ports) (source to spec). The routing of TLP differs based on its type with MWrs and MRds being routed by memory address while Cpl being routed by Requester's BDF identifier.

Today's host CPUs utilize *I/O Memory Management Unit* (IOMMU) to translate physical address to virtual ones in order to securely split I/O peripherals into groups. This unit is crucial in multi-tenant environments (like guests as virtual machines) where only devices in one group can perform P2P transfers between each other. Otherwise, where communication between two groups is required, the data need to be analyzed by the CPU. Without IOMMU a potentially malicious device can write and read from every region in the host system. The unit works hand in hand with the PCIe's *Access Control Services* (ACS) which forces every traffic to pass through the RC even if the devices are connected to the single Switch (this is shown in fig. 1.5). Otherwise, all devices under each Root Port are put to a common IOMMU group which does not play well with the need to decide isolation on per-device basis.

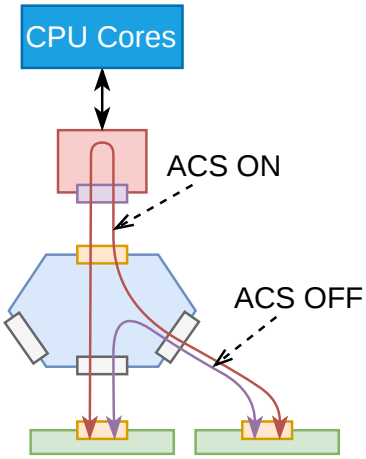


Fig. 1.5: Influence of the ACS mechanism

Considering the test server used in this work, the support for P2P transfers

has been discovered as well as the presence of multiple IOMMUs and enabled ACS on the Root Ports. Since security is not a topic of this thesis nor are the system considerations in the multi-tenant environments, both of these functionalities have been disabled in server's UEFI. Therefore, the PCIe devices communicate using physical addresses.

# 2 Integrated core for PCIe on FPGAs

Modern FPGAs provide hardened IP core that implements the whole PCIe protocol stack (generally implementing a Port) that can be configured either as a Root Port (not as a RC though) or an Upstream Port (mostly for Endpoint, sometimes for Switch on some devices). The IP utilizes high-speed serial transceivers located on the edge of the chip that connect directly to the CEM-compliant Edge connector on the PCB's side. This work utilizes the *Alveo U55C* data-center acceleration card which is equipped with `xcu55c-fsvh2892-2L-e` chip (this corresponds to the serial chip `vu57p`)[5]. The available integrated core called *PCIE4C* allows to configure the interface to run on Gen3 transfer rat on 16 lanes (Gen3x16) or Gen4 transfer rate on 8 lanes (Gen4x8) providing the same throughput but reducing logic utilization. Since two of these cores are provided close to each other and a total of 16 lanes is provided, these cores can split the available lanes and the acceleration card can be configured to run in a *PCIe bifurcation* mode. This mode utilizes a single PCIe link running two physically separated devices (each containing its own set of Functions). Therefore, the acceleration card can reach a cumulative data transfer rate of PCIe Gen4x16.

## 2.1 Interfaces to user logic

Since the described core implement the whole PCIe protocol stack (meaning from Physical to Transaction layer), the user logic does not have to deal with task of the lower layers, such as transaction buffering, flow control credit management, data integrity check, lane deskew or clock recovery. In terms of the PCIe topology, this core implements an Upstream port (specifically as a PCIe Endpoint). The core provides four separate interfaces for the user logic where each follows the *AXI4-Stream* specification. As mentioned before, each PCIe port allows for full-duplex communication, allowing each device to act both as a Requester and as a Completer at the same time. The core contains these separate interfaces exactly in concordance with this behavior which include:

***Requester Request* (RQ)** allows user logic to send requests to other components in the PCIe hierarchy.

***Requester Completion* (RC)** transfers completions for non-posted requests dispatched on the RQ interfaces.

***Completer Request* (CQ)** receives requests from other devices in the PCIe hierarchy and provides them for the user logic.

***Completer Completion* (CC)** is used by the user logic to send responses for non-posted requests received on the CQ interface.

Other than that, the core provides additional interfaces for transferring status information either from the link or from user logic, dispatching and receiving interrupts, sending messages, etc. The PCIe IP maintains its set of status registers as every other PCIe Endpoint that are accessible by an operating system. A separate interface for the Requester side can be used to manage flow control credits but this is not used in this work and the management is left on the PCIe IP.

The width of the AXI4-Stream interfaces depends on the configured transfer rate of the IP. This work implements the DMA engine using 8-lane Gen4 PCIe endpoint which results in a 512-bit wide bus running on 250 MHz. All of the interfaces are running on the same clock that is provided as the 'user clock' output from the IP[6]. Data units communicated on these buses are adjusted TLPs where every is formed as a *TLP header* followed by a payload of user data. The header uniquely identifies a payload within the PCIe topology with a physical address to the reserved space in the host memory or in another PCIe BAR. TLP header has a fixed size of 3 DW (12 B) for RC/CC interfaces and 4 DW (16 B) for RQ/CQ interfaces.

Using wide buses carries a significant drawback when data do not fill whole bus words. For example, transferring 1 B of data using a 512b bus utilizes only about 1.5 % of the bandwidth (1 B of payload plus a 16 B TLP header). To mitigate this inefficiency, the bus word is separated into two segments of equal size. Each segment can contain up to one TLP start and/or one TLP end. This doubles the throughput for packets that fit entirely within a single segment and generally improves throughput when a packet ends in the first segment (the second packet can then begin in the second segment).

# 3 Non-volatile Memory Express

The NVMe standard emerged in 2011 as a way of integration of non-volatile storage into the PCIe domain[1]. The standard emerged as a successor of *SAS* and *SATA* specifications in order to meet the requirements of consumer systems in terms of latency, throughput and scalability that fits with the flash-based storage. All of these standards introduced a queue mechanism where commands (like write and read) are present and processed by the controller of the attached *Non-volatile Media* (NVM). Both, SATA and SAS support one command queue allowing up to 32 and 256 commands respectively. NVMe enhances this mechanism by allowing to instantiate up to $2^{16}$ queues where each can contain up to $2^{16}$ commands. Allowing to have many queues results in hardware-aided parallel access to the NVM where, for example, each CPU core receives its own queue. The schema of the NVMe device is depicted in fig. 3.1. The *NVMe controller* sits on top of a PCIe controller, managing the instantiation of queues and processing of commands. The non-volatile storage is presented as a *Namespace* which is the basic referenceable unit of storage in the NVMe standard[2]. A namespace is split into multiple sectors marked by *Logical Block Addresss* (LBAs) which are at least 512 B in size (which is mostly used one). The controller is equipped with a DMA engine that performs a read and write of raw data between memory regions in the host system. This implies that the NVMe device behaves as a Bus Master and can therefore generate its own MWrs and MRds which is suitable for the P2P communication in the PCIe domain.

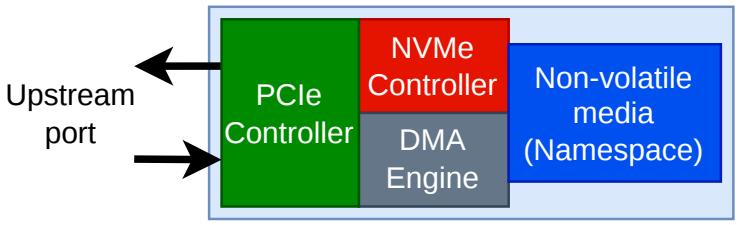


Fig. 3.1: Elementary block scheme of the NVMe device

The command mechanism provides two types of queues that always occur in pairs, the *Submission queue* and the *Completion queue*. A Submission queue contains commands that need to be processed by the NVMe controller which, after a

[2]This thesis works with specification version 1.2b since it describes only its basic concepts and not every currently working NVMe device supports the newest standard. The NVMe specification is designed to be backward compatible though. Using older specifications may be advised for boarding users to understand the basic concepts quickly.

[2]This is a simplified view in the referenced version of the NVMe specification. Later versions define a more complex and fine-grained classification beyond namespaces. Moreover, there can be many namespaces maintained by a single NVMe controller as well as one namespace shared by many controllers.

command's execution, publishes a result of an operation into a Completion queue. These queues are allocated as circular buffers in a host-addressable memory. The most important pair is the *Admin queue-pair* that is used for additional configuration. The controller appears in the PCIe system as a single Function and, initially, the kernel is able to only reach the standard registers in the PCIe Configuration Space and its BARs. The NVMe driver allocates Admin queue-pair and publishes base addresses and sizes of the queues to controller registers located in BAR0.

For data transmission to/from the NVMe namespace, the configuration driver instantiates one or more *I/O queue-pairs* which are used by the consumers/producers of data. Same as for the Admin queue-pair, these queues are allocated circular buffers in host-addressable memory. A fully initialized system with an 8-core processor and an NVMe device is shown in fig. 3.2. The controller can associate multiple I/O Submission queues with one I/O Completion queue in order to save resources (this is not possible for the Admin queue-pair though since only one can be instantiated). Each queue pair is identified by its ID where Admin queue-pair has a fixed ID of 0 and every other I/O queue pair reserves an ID of a higher index.

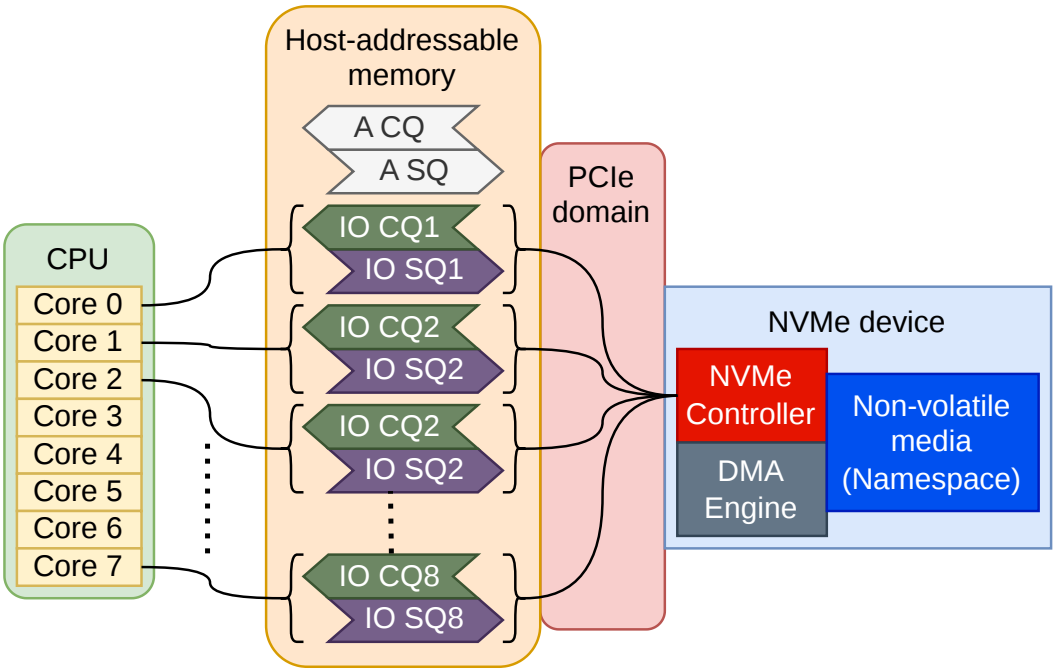


Fig. 3.2: Initialized system with an NVMe device communicating with the host CPU. The indexing of the IO adheres to the standard since the index 0 is reserved for the Admin queue-pair.

## 3.1 Principles of operation

One queue 2 pointers for its operation, a *Head doorbell* and a *Tail doorbell*. A consumer of entries from the queue operates a Head doorbell whereas every producer operates a Tail doorbell. This pair of pointers ensures that a producer does not overwrite unprocessed entries and that a consumer reads only valid ones. A queue is considered empty if values of these pointers are equal, and full if a value of the Tail doorbell is equal the value of the Head doorbell minus one modulo a size of the queue as is shown in figs. 3.3 and 3.4.

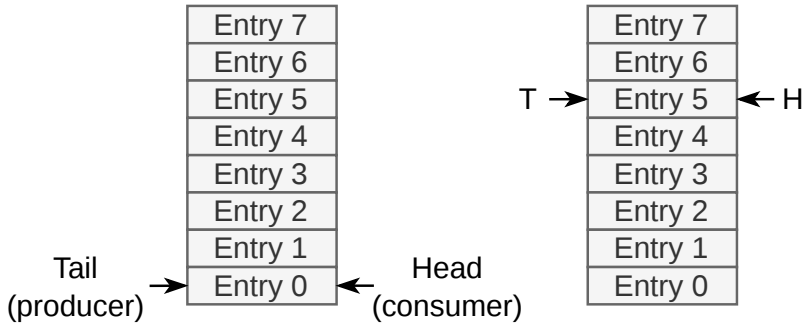


Fig. 3.3: Two examples of an empty queue

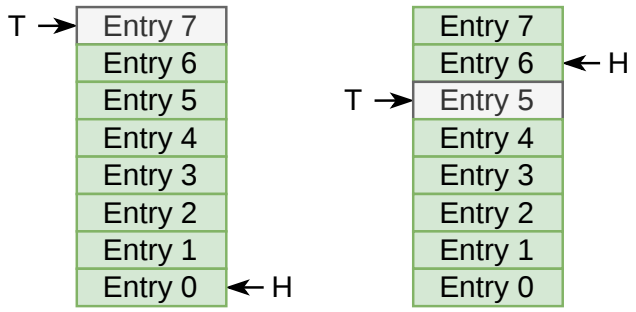


Fig. 3.4: Two examples of a full queue

A master submits a command to a submission queue by using a template for *Submission Queue Entry* (SQE) described further. Upon storing such entry in the queue, the master updates the *Submission Queue Tail Doorbell* (SQTDBL) pointer in controller registers (in BAR0) in order to signalize the controller that new command has been submitted. This is followed by the controller fetching and executing the command followed by dispatching a completion into a completion queue that follows the *Completion Queue Entry* (CQE) template. If a command operates with additional data (like reading/writing data from/to the NVM or fetching commands from a submission queue), the controller engages the DMA engine on the device. To detect a received completion of a command, the master does not have to read the

*Completion Queue Tail Doorbell* (CQTDBL) value (since this is not available and remains internal to the NVMe controller anyways). Rather, every CQE contains a *Phase tag (P)* that indicates that a position in the Completion queue contains a valid entry.

### 3.1.1 Command submission

The SQE is created by a master by following a format in fig. 3.5 and it uniquely defines a command within a single NVMe controller. The entry has a length of 64 B (16 DW) with necessary information for a controller to process a command. There are two sets of commands defined by the NVMe specification, namely *Admin Command Set* and *NVM Command Set*[3]. The mostly used used commands in this thesis on runtime are read and write from/to the NVM that are described in the following paragraph.

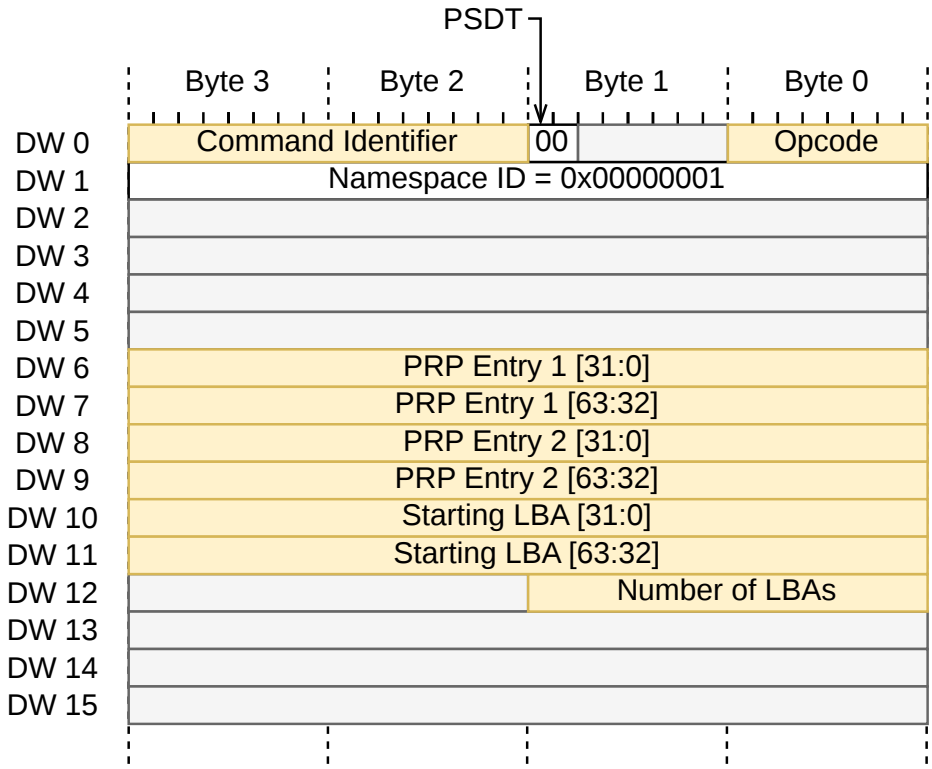


Fig. 3.5: Format of Submission Queue Entry. The greyed-out fields are either reserved or unused within this thesis.

An SQE begins with the *Opcode* (OPC) field that distinguishes between different types of commands, the *PSDT* field that specifies that *Physical Resource Pages* (PRPs) are going to be used as copy buffers and the *Command Identifier* (CID) field that presents a unique identifier when combined with the Submission Queue ID. This combination serves as a tag to match the dispatched command with a received

completion. The first command DWord is followed by *Namespace Identifier* (NSID) to index a specific namespace that is managed by the adjacent NVMe controller (the index is 1-based, meaning that the first namespace receives NSID 1). The following *Data Pointer* (DPTR) field serves as a pointer to the host-addressable memory either from which data need to be fetched (as for the write command) or to which the data are copied from the NVM (as for the read command). The data pointer is split into two PRP entries depending on how many memory pages contain the operated data in host memory, thus how big the transfer is going to be. If the transfer fits into one page, only *PRP Entry 1* (PRP1) is used and it points to the start of the user data. In case a transfer spans over 2 memory pages, the *PRP Entry 2* (PRP2) is used as well to point to the second page. If a transfer spans more than 2 pages, the PRP2 changes its meaning to point to the *PRP List* which contains pointers to every other page that contains to the user data (meaning from the second one onwards). The following two Dwords (DW10 and DW11) contain the *Starting LBA* (SLBA) field in the namespace the last usable field in DW12 contains the *Number of Logical Blocks* (NLB) to copy. This size determines a size of a transfer for a currently processed command. The size of a transfer is limited by the NVMe controller and every master has to consult controller parameters in order to not submit larger transfers. When larger data chunks are required than a transfer can process, multiple commands have to be submitted.

### 3.1.2 Command completion

After a command is processed the controller dispatches a CQE to the Completion queue to notify the master about the status of a command. The template of every entry is shown in fig. 3.6. A CQE is at least 16 bytes in size. The first DWord is used by some commands to publish command-specific information (the Read and Write commands do not use them though) while the second DWord is reserved. An NVMe controller indicates the internal value of *Submission Queue Head Doorbell* (SQHDBL) through every completion entry which notifies the transmitter of commands about how many commands have been read by the controller (thus providing a backpressure). The *SQ Identifier* (SQID) and CID fields serve a unique identifier of a command in the communication chain. The SQID is present because the NVMe specification allows to associate multiple Submission queue with one Completion queue (this applies for one NVMe controller though since every

[3]In terms of more recent specification revisions, these sets use the *memory-based* transport which differs from the *message-based* transport. While the first type of transport is typical for PCIe, newer specifications allow to attach NVM devices to other fabrics, like RDMA or Ethernet, that use the second type of transport. These are part of the *NVMe over Fabric* (NVMe-oF) standard published separately.

controller maintains its own set of queues). A *Phase Tag (P)* field follows and its value indicates that the current position in the Completion queue is valid (meaning the position to which *Completion Queue Head Doorbell* (CQHDBL) points to). The value that is considered valid is flipped upon every CQTDBL rollover, starting as logical 1 for the initial pass through queue (i.e. after queue initialization), then flipping to 0 when the pointer rolls back to 0, then to 1 again and so on.

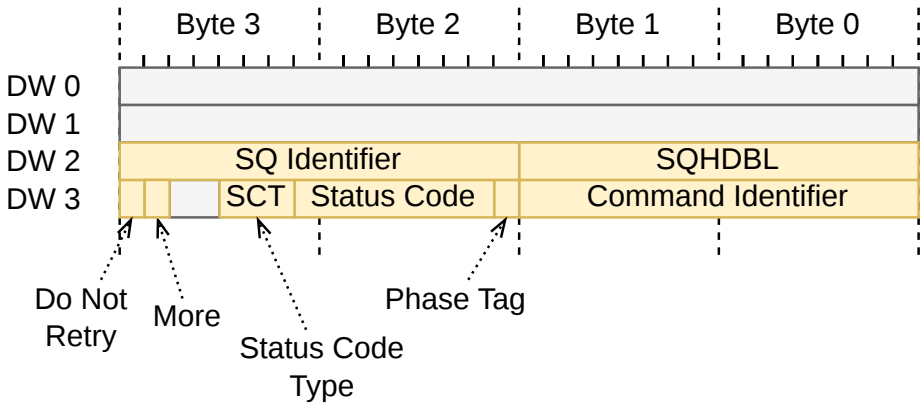


Fig. 3.6: Format of Completion Queue Entry. The greyed-out fields are either reserved or unused within this thesis.

The last bits of a CQE are occupied by status information about the successful/unsuccessful execution of a command. The *Do Not Retry* (DNR) bit indicates that if the same command is going to be resubmitted, it is expected to fail. The submitter has to wait till this bit clears to 0 in order to receive valid data. The *More (M)* bit indicates that a completion for a command is going to be composed out of multiple CQEs. A more specific information about a command's status can be found in the *Status Code Type* (SCT) and *Status Code* (SC) fields. A NVMe controller provides coarse-grained category of status within the first field whereas a more fine-grained in the second one. Here are some examples of possible status codes received:

- A successful completion is indicated by the SCT field set to 0h (i.e. *Generic Command Status*) and the SC field set to 00h (i.e. *Successful Completion*).
- When a submitter dispatches an NVMe command with a specified LBA range that exceeds the capacity of the Namespace, the SCT field is set to 00h and the SC field to 80h (i.e. *LBA Out of Range*).
- Submitting two NVMe Write/Read commands with the same CID results in a completion status where SCT is set to 1h (i.e. *Command Specific Status*) and SC set to 80h (i.e. *Conflicting Attibutes*).

### 3.1.3 Ordering

An important note has to be made with regards to ordering of submitted commands. An NVMe controller arbitrates between its own set of submission queues in a round-robin fashion and can fetch multiple commands out of every queue[4]. The order of execution where many commands are fetched is not defined by the NVMe specification and can be arbitrary. This is the reason why a tagging of commands using the CID field is important so the submitter can bound a completion to a dispatched command. The order of execution can therefore be derived from the order in which the completions for multiple commands have been received. If a strict ordering of commands is required, a submitter needs to implement its own for that. However, by the means of the NVMe specification, there are two ways of how to ensure ordering. The standard allows to use *Fused operations* in order to force ordering of two subsequent commands. Such two commands can be executed atomically, resulting in the success only if both of the commands succeed. However, this feature is optional and, therefore, implementation specific. The second way that is always available, is to submit only one command to the submission queue and wait for its completion before submitting another commands. This can be guarded by the submitter itself or by initializing the submission queue with the size of 2 (since this can contain at most 1 valid SQE) and using the blocking by the doorbell pointers. The second way is used in the implementation proposed by this thesis.

## 3.2 Multi-pathing

Since multiple queues can be initialized for a single NVMe controller, multiple submitters can execute commands on the controller. Since the proposed implementation in this thesis introduces a direct NVMe device access from an FPGA acceleration card using the PCIe P2P transfer, it contains a potential to access multiple NVMe devices (the 1-to-M topology) as well as to access multiple NVMe devices from multiple acceleration cards, thus implementing an M-to-N topology[5] as seen in fig. 3.7. This feature that allows to access one NVMe device from multiple masters is sometimes called *multi-pathing*.

[4] Advanced devices can implement a Weighted Round-robin or some vendor-specific mechanism.

[5] This topology, however, is only virtual since it is made only by addressing and not by physical connections. When performance (like throughput or latency) is critical, it should be adapted on the underlying PCIe topology which is organized as a tree by design.

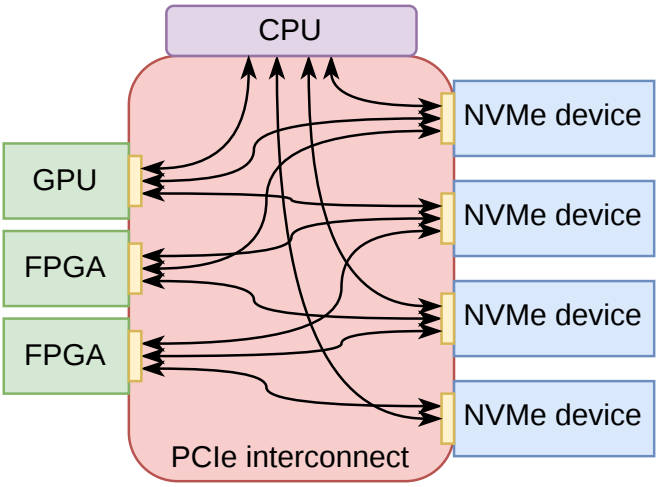


Fig. 3.7: Example topology with M:N mesh network

# 4 DMA Iuventus

Since this thesis proposes a solution that is able to access the filesystem located on an NVMe device that is shared between the host OS and an FPGA accelerator a specific IP core is needed on the accelerator's side. Without this, the user logic inside an FPGA would deal with raw PCIe data not only from the adjacent NVMe device but from the host system as well. Therefore, the *DMA Iuventus* IP core has been developed to process requests between an accelerator and an NVMe device. The DMA description has been retained to highlight the mechanism of the direct copying of data to the adjacent storage without a host CPU facilitating such transfers (this is sometimes called *Host bypassing*). This IP sits on the integrated block for PCIe of the FPGA to accept NVMe-specific traffic alongside with a logic that processes transactions to *Configuration and Status* (C/S) registers.

## 4.1 TLP Headers

The DMA IP utilizes 3 of the 4 interfaces provided by the integrated block for PCI Express, namely RQ, CQ, CC. Each on of these uses its own template of the TLP header that the DMA core needs to create or process. The RQ header has a size of 4 DW and its format is depicted in fig. 4.1. The *Address* field is aligned with respect to DWords since its two bottom bits are reserved[1]. For MWrs, the *Dword count* field specifies the size of the payload following the header, whereas, for MRds, it specifies the required amount of data that a Completer needs to send in its Completion. The type of a request is defined in the *Req type* field. The *Func* field informs the PCIe IP which Function generated the request (more on that later). The last used field is the *NoSnoop* bit which instructs the the host CPU that the underlying address space the transaction is targeted to, does not require a hardware-enforced cache coherency[2]. This means that the address space is not shared by any other device in the system. The RQ interface is used to dispatch SQTDBL and CQHDBL updates to the NVMe registers.

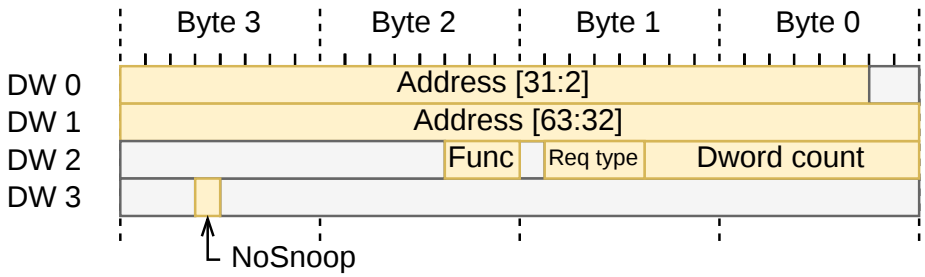


Fig. 4.1: Format of the header used on the Requester Request interface

The CQ header (Figure 4.2) follows a similar format as the RQ header where the user logic acts as a PCIe Completer. The *Target function* field indicates to which Function a request is targeted. When multiple BARs are used, the core puts index of the specific one in the *BAR ID* field. The size of a targeted BAR is determined by the *BAR Aperture* which serves as a mask for the address (for example, the value of 12 signifies that the BAR has a size of 4 KiB and, therefore, the address bits [63:12] can be ignored). Other fields of this header are either reserved or used to create a response for MRd.

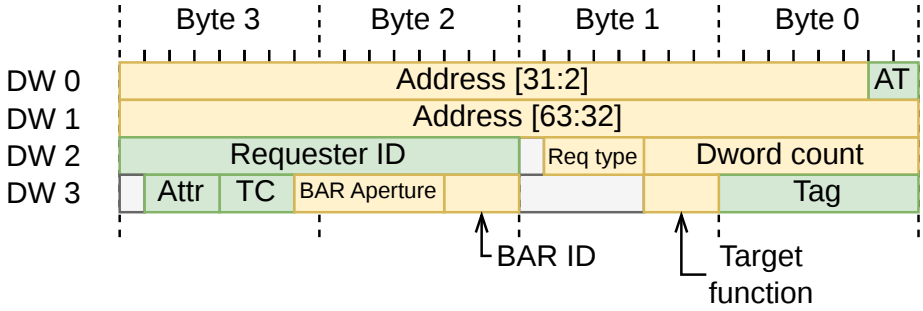


Fig. 4.2: Format of the header used on the Completer Completion interface. The yellow fields are used by the DMA Iuventus core itself whereas the green ones are only used to create the CC header.

For posted requests, such as MRd, the response is dispatched on the CC interface by using the 3 DW TLP header in fig. 4.3. The header may be followed by a payload. The *Address* is a byte-level address of the first byte of the memory block being transferred. The size of the payload is specified in the *Dword count* field. Because the CQ header contains only DWord-aligned address, the DMA logic needs to calculate the byte-level offset by reading the *TUSER* signal of the AXI-Stream bus on the CQ interface which contains the *First Byte Enable* (FBE) as a mask of valid bytes int the first DWord of the payload. The byte offset is then adjusted by factoring initial valid/invalid bytes in the calculation. There is another field used, namely the *Last Byte Enable* (LBE) which is located in the same AXI-Stream signal. This mask serves for the last DWord of the payload to distinguish valid bytes. An exact byte length of the payload can be calculated by using the Dword count, FBE and LBE fields.

Each completion transfers a status of either *Successful Completion*, *Unsupported Request* or *Completer Abort* in the *Status* field. Similar to the Target Function field in the RQ header, there is an index of a completing Function. Other than that, there are multiple fields that need to be copied from the Requester TLP header, such as *Address Translation* (AT), *Requester ID*, *Tag*, *Transaction Class* (TC) and

[1]The x86_64 architecture addresses its physical memory space with resolution to bytes.

*Attributes* in order that the Completion is correctly routed to the Requester and then correctly classified within its internal system. This applies especially for the Tag field which marks all Completions for a specific Request. A Completion can be dispatched in multiple TLPs (especially if the MRRS of the Completer is bigger that the MPS value) which is controlled using the *Byte count* field. The value indicates the remaining amount of bytes that are needed to complete a request including the length of the current payload. The last Completion TLP has hast the value of *Byte count* equal to the byte-length of its payload.

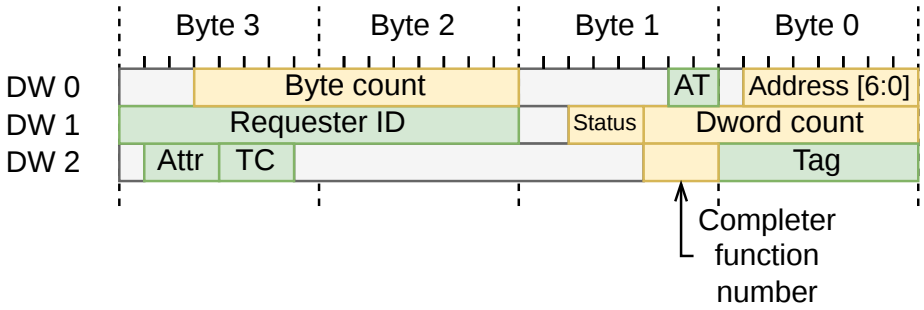


Fig. 4.3: Format of the header used on the Completer Completion interface. The yellow fields are generated by the DMA Iuventus core itself whereas the green ones are copied from the CQ header.

## 4.2 Software configuration

This thesis utilizes the *NDK-FPGA* framework developed by *CESNET, a.l.e.*[7]. This framework serves as a development platform for datacenter-grade FPGA acceleration cards providing stable communication infrastructure and allowing to instantiate a custom user logic. In order to provide the software control, the *NDK-SW* package is delivered besides the previous framework[8]. This package allows to access internal C/S registers of various components within the FPGA logic from userspace. Additionally, this thesis utilizes the *Storage Performance Development Kit* (SPDK) framework in order to correctly configure an NVMe device and establish a P2P communication chain between an accelerator card and the storage drive. SPDK is a userspace framework to create high-performance applications where drivers are running in the userspace[9]. This enables fast prototyping without the need to operate with kernel modules which can cause fatal system failures if not done carefully.

A device that is normally maintained by a kernel driver can only be accessed by a defined API that usually utilizes the *sysfs* filesystem or, when used in custom applications, the *ioctl* calls with defined flags. Upon every modification of such driver, it needs to be compiled and inserted into the kernel not to mention the

limited possibilities of debugging. This mechanism, however, prevents malicious applications from accessing the MMIO resources directly. In order to move all of the control to the userspace, a device needs to be unbound from a particular driver and bound to a new driver[2]. The Linux kernel stack provides two "dummy" drivers that serve the purpose of this thesis. If IOMMU is disabled, the system operates only with physical addresses, the device needs to be bound to the `uio_pci_generic` driver, whereas, if enabled, the system operates with *I/O Virtual Addresss* (IOVAs) and the `vfio-pci` driver needs to be used. Both of these drivers exposes devices' BARs to the userspace where they can be managed by standard memory operations. Since using the former approach with disabled IOMMU, the application need to be run as the root user.

### 4.2.1 Memory partitioning

The fig. 3.2 presents a standard customer system with a main NVMe drive and an instantiated set of queues for every core on the processor. The queues that are created for this purpose are normally stored in the host memory where the 2 buffers get allocated for every queue pair as can be seen on figure XXX. The data buffers are dynamically generated since values of PRP1 and PRP2 can vary with every dispatched command, therefore conforming to the very principle of the DMA where data do not have to be copied to special "copy" buffers. The values of these fields can therefore point directly to user data in the host memory.

Since this work wants to establish the P2P communication chain between an FPGA accelerator and an NVMe drive which bypasses the host system the mapping of queues as well as copy buffers has been moved entirely to the PCIe BARs of the accelerator. Such system is depicted in figure XXX. Because the mapping of PCIe BARs of the accelerator is constant the PRP1 and PRP2 fields are always pointing either to the *Read Buffer* (which contains data that the NVMe needs to fetch as a part of the write command)) or to the *Write Buffer* (which is a reserved space where the NVMe uploads its data as a part of the read command). These buffers are allocated in BARs 3 and 2 respectively. The queue pair is located in the first two BARs where Submission Queue is located on BAR 0 and Completion Queue on BAR 1. By mapping all of these data structures to the PCIe device, the copying of user data through host memory is entirely removed.

The *NDK-SW* framework provides its own kernel driver in order to operate on the PCIe device and to access its C/S registers. However, in order to establish a P2P communication chain, the NVMe device as well as the FPGA accelerator need to be unbound. The unbinding simplifies the configuration of each device

[2]For concrete steps of how to unbind a device, check the chapter A

which directly exposes its BARs' physical addresses and its PCIe Configuration Space. The NVMe device can be entirely unbound since no original kernel driver is used and everything considering a device initialization is handled by the SPDK framework. On the other hand, with regards to the FPGA accelerator, a different approach had to be chosen, namely to create a second PCIe Function on the device. This allows two drivers in the host system to operate on one device while keeping separate Configuration Space for each one of them. Having this, the access to the configuration registers of the FPGA's user logic can be preserved while a custom driver for the P2P communication chain can be used besides that. Since there is only one PCIe IP per one physical connector on the FPGA chip, all of the requests pass through same interfaces to the user logic and have to be split according to the function index and the BAR ID.

### 4.2.2 Queue initialization

This step relies on the initial configuration of the controller done by the SPDK framework. Especially, this involves the allocation and assignment of Admin queue-pair whose addresses are stored in the controller's registers (remember that these registers reside in the PCIe BAR of the NVMe drive). The NVMe controller has two ways of configuration, either through its registers (works every time) or through its Admin queue pair (after it gets allocated) using the *Set Features* command. The following list describes the specific case for the solution presented in this thesis and not the general one presented in the specification.

1. The *Controller Capabilities* (CAP) register indicates the maximum supported size of its queues and the requirement if queues need to be physically contiguous.
2. The host allocates space for the Admin Completion and Submission queues with its preferred size and writes these to the *Admin Queue Attributes* (AQA) register. Their base addresses are then written to the *Admin Completion Queue Base Address* (ACQ) and *Admin Submission Queue Base Address* (ASQ) registers. The Admin queue pair has an implicitly reserved Queue ID 0.
3. The host submits a *Set Features* command with the *Number of Queues* attribute set to the desired amount of I/O Submission and I/O Completion queues. The controller responds with the amount of queues that it supports which can be equal or lower than the number of the queues requested.
4. The Queue ID 1 is reserved for the queue pair residing in the FPGA accelerator. This reservation is not only internal to the SPDK framework but also necessary for correct registration of queues on the NVMe controller to keep consistent state in the whole stack when more queues need to be added[3].

5. The host allocates I/O Completion queue in the BAR 1 of the FPGA accelerator and clears its content. The clearing is necessary since the DMA Iuventus logic could otherwise process past entries that have a *Phase Tag* field set to 1 (which is the initial value that is considered valid after the queue initialization).
6. The *Create I/O Completion Queue* command is submitted in order to register the queue on the controller.
7. The host allocates an I/O Submission queue in the BAR 0 of the FPGA accelerator and submits the *Create I/O Completion Queue* command to register this queue on the controller.
8. The Queue ID 2 is reserved for the queue pair for the host to create and maintain a filesystem on the NVMe storage.
9. The host allocates regions in the host memory that are going to hold its I/O queue pair. The Completion queue is cleared in the same way as in the case of the FPGA accelerator.
10. The queues are registered in the same order as previously, first the Completion queue, then the Submission queue.
11. The system is now ready to operate and write/read data to/from the non-volatile storage.

## 4.3 Communication model

The P2P chain between the FPGA accelerator and the NVMe drive has been partially established by creating the accelerator-owned queue pair. In order to perform the full P2P communication, the host allocates the BAR 2 and BAR 3 of the accelerator and publishes its physical addresses to the Iuventus module so the submitted commands to the NVMe target the internal buffers of the FPGA.

### 4.3.1 Read command

The read command to the NVMe consists of several steps as seen in figure XXX. The command is submitted by writing one SQE data structure to the Submission queue and updating the corresponding SQTDBL (in case of the accelerator queue pair, this would be SQ1TDBL, whereas for the host-side, the SQ2TDBL). Afterwards, the controller sends a MRd to the Submission queue to fetch the command. The requester (either the DMA Iuventus module or the host software) responds with one Cpl TLP packet. The controller then proceeds to execute the command. The read command targets either the accelerator's or the host's *Write buffer* which is specified

[3]Remember that the Queue ID is used in the CQE as a part of a command's unique identifier since multiple I/O Submission queues can share one I/O Completion queue

in command's PRP1 and PRP2 fields. The NVMe controller can send the data in one or multiple MWrs. Finally, to signalize the end of command execution, a CQE is written to the Completion queue. As seen in the same figure, the requester can submit multiple commands which are then processed on the controller. To inform the controller that all completions have been read from the Completion queue, the requester updates the value of the corresponding CQHDBL in controller's registers (similar to the SQTDBL, the accelerator updates the CQ1HDBL value whereas the host the CQ2HDBL value).

If it needs to be used, the *PRP List* has been created in the host memory as part of FPGA device initialization. This list resides in the host memory for every requester regardless if the accelerator or the host engages in the communication. The values of its elements for the FPGA accelerator are constant (that is because the mapping of BARs does not change) whereas for the host side, it is created on the go[4].

### 4.3.2 Write command

The write command contains additional steps with comparison to the read command as seen in fig XXX. The NVMe controller fetches the command in a similar manner, by receiving an SQTDBL update, dispatching the MRd to the requester and receiving completions with SQE data. Afterwards, as a part of command execution, the controller dispatches one or multiple MRds to requester's *Read buffer* to fetch the data that are received consequently in one or multiple Cpls. The end of a command execution is marked by the write of a CQE with the status of the execution. Similarly, the controller can fetch multiple commands and process them as a batch, dispatching the completion every time. The requester then, after processing the CQE, updates the corresponding value of CQHDBL. Optionally, the requester can issue a *Flush* command to permanently store the data on the NVM which can be seen in the same figure as well. This command can be issued on devices that use a separate write cache. It is not necessary to explicitly use this command unless synchronization between two requesters is required. The flushing can be left on the automatic mechanism or the controller which also flushes the whole content of a cache when it shuts down.

[4]The API of the SPDK framework already contains built-in functions to dispatch read/write commands to the NVMe device and the maintenance of the PRP List is, therefore, left to its own devices.

## 4.4 Hardware implementation

## 4.5 Evaluation

# Conclusion

Thesis conclusion.

# Bibliography


[1] Ravi Budruk Mike Jackson. *PCI Express Technology*. Mindshare, Inc., 2012. ISBN: 978-0-9836465-2-5.

[2] *PCI Express Base Specification*. Revision 4.0, Version 1.0. PCI-SIG, 2017.

[3] Intel Corporation. *ATX Version 3.0 Multi Rail Desktop Platform Power Supply*. Version 2.0. 2022. URL: https://www.cybenetics.com/attachs/52.pdf. (accessed: 4.3.2026).

[4] Wikimedia Commons. *Example of PCI-E x4, x16, x1 and 2nd x16*. URL: https://commons.wikimedia.org/wiki/File:PCI-E_%26_PCI_slots_on_DFI_LanParty_nF4_SLI-DR_20050531.jpg. (accessed: 23.10.2025).

[5] Inc. Advanced Micro Devices. *Alveo U55C Data Center Accelerator Cards Data Sheet (DS978)*. Version 1.3. 2023. URL: https://docs.amd.com/viewer/book-attachment/jBy7Lf0ukySp4TFAfKhJJQ/aj6Aw4v67dCW~TvHEOe5UQ-jBy7Lf0ukySp4TFAfKhJ (accessed: 23.10.2025).

[6] Inc. Advanced Micro Devices. *UltraScale+ Devices Integrated Block for PCI Express v1.3*. Product Guide. 2025. URL: https://docs.amd.com/viewer/book-attachment/tCJDwogjyJ9_CE2Q_px23A/V1qgleVOyfTzj42ioh4SWw-tCJDwogjyJ9_CE2Q_px23A. (accessed: 23.10.2025).

[7] a.l.e. CESNET. *Network Development Kit (NDK) for FPGA cards*. URL: https://github.com/CESNET/ndk-fpga. (accessed: 23.10.2025).

[8] a.l.e. CESNET. *Linux driver and SW tools for Network Development Kit (NDK)*. URL: https://github.com/CESNET/ndk-sw. (accessed: 4.3.2026).

[9] Ziye Yang et al. "SPDK: A Development Kit to Build High Performance Storage Applications". In: *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 2017, pp. 154–161. DOI: 10.1109/CloudCom.2017.14.

# Symbols and abbreviations

| | |
|---|---|
| **AT** | Address Translation |
| **ACS** | Access Control Services |
| **ACQ** | Admin Completion Queue Base Address |
| **AQA** | Admin Queue Attributes |
| **ASQ** | Admin Submission Queue Base Address |
| **BAR** | Base Address Register |
| **DNR** | Do Not Retry |
| **DPTR** | Data Pointer |
| **CAP** | Controller Capabilities |
| **CC** | Completer Completion |
| **CEM** | Card Electromechanical |
| **CID** | Command Identifier |
| **Cpl** | Completion |
| **CQ** | Completer Request |
| **CQE** | Completion Queue Entry |
| **CQHDBL** | Completion Queue Head Doorbell |
| **CQTDBL** | Completion Queue Tail Doorbell |
| **C2H** | Card-to-Host |
| **C/S** | Configuration and Status |
| **DLLP** | Data-link Layer Packet |
| **DMA** | Direct Memory Access |
| **D2D** | Device-to-Device |
| **D2H** | Device-to-Host |
| **EP** | Endpoint |

| | |
|---|---|
| **FBE** | First Byte Enable |
| **FPGA** | Field-programmable Gate Array |
| **H2C** | Host-to-Card |
| **H2D** | Host-to-Device |
| **IOMMU** | I/O Memory Management Unit |
| **IOVA** | I/O Virtual Address |
| **LBA** | Logical Block Address |
| **LBE** | Last Byte Enable |
| **MDTS** | Maximum Data Transfer Size |
| **MMIO** | Memory-mapped I/O |
| **MPS** | Maximum Payload Size |
| **MRd** | Memory Read Request |
| **MRRS** | Maximum Read Request Size |
| **MWr** | Memory Write Request |
| **NIC** | Network Interface Card |
| **NLB** | Number of Logical Blocks |
| **NSID** | Namespace Identifier |
| **NVM** | Non-volatile Media |
| **NVMe** | Non-volatile Memory Express |
| **NVMe-oF** | NVMe over Fabric |
| **OPC** | Opcode |
| **PCI** | Peripheral Component Interconnect |
| **PCIe** | Peripheral Component Interconnect Express |
| **PRP** | Physical Resource Page |
| **PRP1** | PRP Entry 1 |

| | |
|---|---|
| **PRP2** | PRP Entry 2 |
| **P2P** | Peer-to-peer |
| **RC** | Root Complex |
| **RC** | Requester Completion |
| **RQ** | Requester Request |
| **SC** | Status Code |
| **SCT** | Status Code Type |
| **SLBA** | Starting LBA |
| **SPDK** | Storage Performance Development Kit |
| **SQE** | Submission Queue Entry |
| **SQID** | SQ Identifier |
| **SQHDBL** | Submission Queue Head Doorbell |
| **SQTDBL** | Submission Queue Tail Doorbell |
| **SSD** | Solid-state Drive |
| **TC** | Transaction Class |
| **TLP** | Transaction Layer Packet |

# List of appendices

**A SPDK Framework configuration** **57**

# A SPDK Framework configuration

The framework can be downloaded from the fork of its GitHub repository (cite this) and switching to branch XXX.